



Telemetry Dashboard

Software Design Document

Authored by: Ryan Tengan

Table of Contents

Revision History.....	2
Interfaces.....	3
Data Acquisition.....	3
Pipeline.....	3
SocketCAN.....	3
Logging.....	4
MQTT Broker.....	4
Frontend.....	5
SvelteKit.....	5
Functional Capabilities.....	5
Backend.....	6
FastAPI.....	6
File Processing.....	6
Database.....	6
Build and Verification.....	8
Build Systems and Package Management.....	8
CMake.....	8
Vcpkg.....	8
Node Package Manager.....	8
Verification.....	8
Test Scripts.....	8
Helpful Links.....	9

Revision History

Version	Date	Author	Comments
1	03/12/2026	Ryan Tengan	Initial Document

Interfaces

Data Acquisition

Pipeline

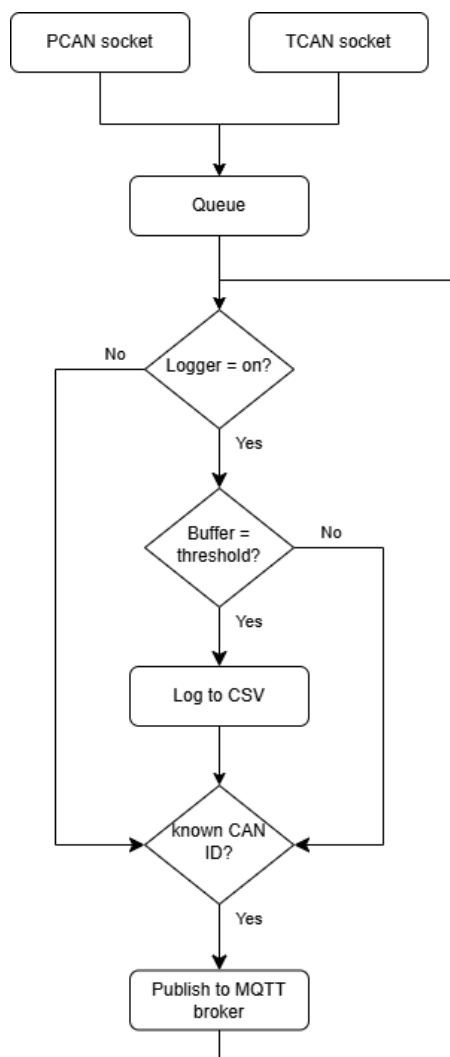


Figure 1: CAN Bus Data Pipeline

SocketCAN

In Linux, SocketCAN is a kernel networking subsystem that exposes CAN buses as network interfaces, allowing applications to send and receive CAN frames using standard socket APIs.

Telemetry Dashboard: Software Design Document

To interface with the vehicle's two CAN buses (PCAN and TCAN), the Data Acquisition (DAQ) service uses two dedicated threads. Each thread binds a CAN socket to its respective bus and enqueues received frames into a lock-free queue, preventing contention and eliminating the risk of deadlocks.

Logging

The MQTT broker is used to control data logging, ensuring clean separation of individual data logs.

MQTT Topic	Description
"logger/status"	Used to let clients know when the Pi is logging CAN bus data. When the DAQ service is started, the status is initialized as "off" and published to the MQTT broker (retained message, quality of service (QoS) level 1).
"logger/control"	Used for publishers to start ("on") or stop ("off") logging. • Start logging: a data log is created in the "data/can/intake" directory • Log filename format: FE13CAN_Ymd_HMS.csv • Stop logging: data log is moved to "data/can/ready", indicating that the log is ready to be processed

MQTT Broker

Once a CAN frame is received from the queue, it is serialized to JSON and published to the MQTT broker:

- **MQTT:**
 - Topic: "can/frame"
 - QoS 0 ("at most once")
- **JSON:**
 - CAN ID
 - Timestamp
 - Message field 0
 - ...
 - Message field N

Telemetry Dashboard: Software Design Document

Frontend

SvelteKit

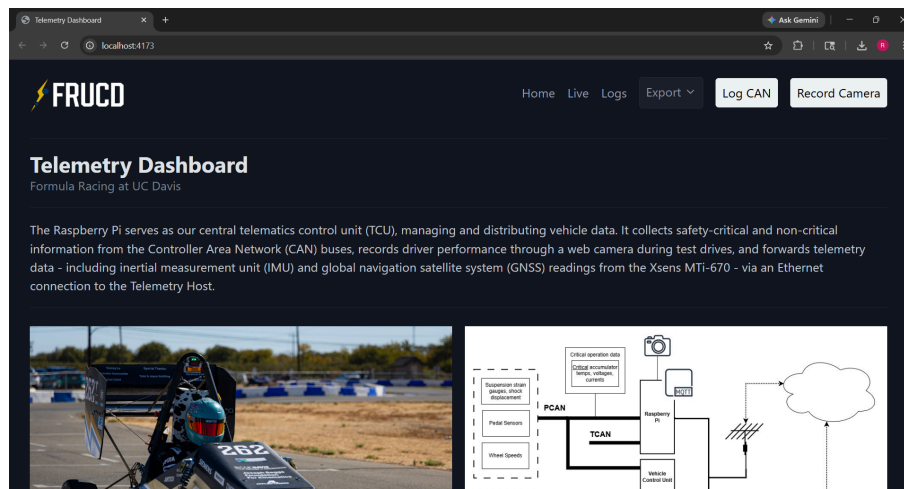


Figure 2: Telemetry Dashboard GUI

In order to minimize work done by the Raspberry Pi, the frontend was built using a single-page application (SPA) using the SvelteKit web framework. This offers:

- **Fast development time:** Svelte's simple syntax and component-based structure let you build UI features quickly without boilerplate.
- **Reactivity:** State changes automatically update the DOM; you don't need to manually manipulate elements.
- **Minimal Bundle Sizes:** Svelte compiles components to plain JavaScript with no heavy runtime, resulting in small JS files that load quickly, perfect for low-power devices like a Raspberry Pi.

Functional Capabilities

- **Live Data:** Monitor real-time vehicle information and camera feeds.
- **Logging:** Manage data recording and download multiple logs at a time.
- **Analysis:** Enable clients to review and analyze vehicle data.
- **Communication:** Control and transmit CAN messages from the Raspberry Pi.

Backend

FastAPI

The backend was built using the Python FastAPI framework. This offers:

- **Rapid development time:** FastAPI's modern Python makes it quick to develop robust APIs.
- **Asynchronous performance:** Native support for async functions allows concurrent handling of file watchers, data processing, and client requests without blocking the server.
- **Lightweight and efficient:** FastAPI is minimal and highly performant, making it well-suited for the Raspberry Pi while still supporting complex operations.

File Processing

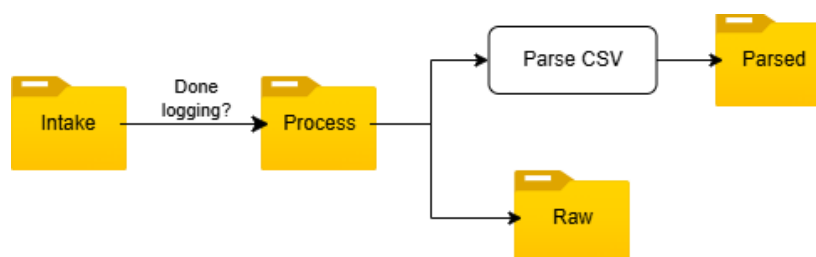


Figure 3: File Processing Pipeline

To prevent server stalling, file processing is offloaded to a background task. Within this task, a file watcher monitors for new data logs and parses the files accordingly. The processing pipeline includes four main stages:

- **Intake:** CAN bus data is continuously logged by the DAQ service.
- **Process:** Once a data log is complete, the background task detects any new files added to the designated folder.
- **Parsed:** The background task parses the CSV, decoding messages according to the formats defined in the CAN database (DBC) files, and stores the results in the parsed folder.
- **Raw:** After parsing, the original CSV is moved to the raw folder for archival.

This file organization allows for efficient data management and clear separation between raw and processed logs.

Database

Because multiple clients may want to access logged vehicle data concurrently, an SQL database is required. In addition to the raw CAN frame data, the system must support queries based on:

- Timestamps

Telemetry Dashboard: Software Design Document

- Driver (useful for individual performance analysis)
- Vehicle (FE12, FE13, etc.)
- Filename
- Location: Blue Max, Thunderhill, Michigan, etc.

Since we are working with high volumes of timestamped data, a time-series database (TSDB) is well-suited for our application. TSDBs are optimized for:

- Storing and indexing time-ordered data
- Enabling efficient range queries
- Time-window aggregations
- Automatic handling and interpolation of missing timestamps
- High write throughput

Build and Verification

Build Systems and Package Management

CMake

CMake is used to build the C++ DAQ service. It manages compiler settings, links required libraries, and generates build files for the Raspberry Pi, ensuring consistent and reproducible builds.

Vcpkg

Vcpkg is used to manage third-party C++ dependencies required by the DAQ service. It integrates with CMake to automatically install and configure external libraries, improving portability and version consistency.

Node Package Manager

Node Package Manager (npm) is used to manage dependencies and build tooling for the frontend. It installs required JavaScript packages and compiles the Svelte application into optimized static files, which are then served by the FastAPI backend.

Verification

Test Scripts

Virtual CAN messages are used to simulate the PCAN and TCAN buses in order to validate the application's handling of CAN data. When paired with command-line tools provided by Eclipse Mosquitto, this enables controlled end-to-end testing of data ingestion, message handling, and logging workflows.

For example, one test script (replay.py) reads a previously recorded CAN data log and sends the raw messages to a virtual CAN interface, allowing verification that the code behaves correctly under real-world conditions.

Helpful Links

- [SocketCAN](#): Linux networking stack for interfacing with CAN buses
- [Mosquitto MQTT Broker](#): Message broker that implements MQTT
- [Svelte](#): Compiler that transforms components into vanilla JS at build time
- [FastAPI](#): High-performance Python web framework for building APIs
- [Vcpkg](#): C/C++ package manager that simplifies acquiring and managing libraries
- [CMake](#): Generates native build files (e.g., Makefiles) to compile, test, and package software
- [npm](#): Offers open-source libraries and a command-line utility to manage dependencies